



[ACT] S-12

Recuperación de contraseñas accesible y segura

Alumno: Oscar Espinoza Landeta

Docente: José Miguel Carrera Pacheco

Fecha: 23 de marzo de 2025

Materia: Desarrollo Web Profesional — 8° Semestre

1. ¿Qué se investigó?

Para diseñar un flujo de recuperación de contraseñas seguro y accesible fue necesario investigar cuatro áreas fundamentales:

1.1 ¿Qué es un token de recuperación?

Un token de recuperación es una cadena generada criptográficamente que actúa como credencial temporal y de un solo uso. Permite autenticar al solicitante sin requerir la contraseña actual, vincula la solicitud a un usuario y a un momento específico, y queda destruido tras su primer uso.

En este proyecto se generó con `crypto.randomBytes(32)` de Node.js, produciendo 32 bytes de entropía criptográficamente segura (token hex de 64 caracteres). La probabilidad de colisión es $1/2^{256}$, prácticamente imposible de predecir por fuerza bruta.

1.2 ¿Por qué deben expirar los tokens?

Un token sin expiración equivale a una contraseña permanente almacenada en un enlace. Los riesgos principales son:

- **Canal comprometido:** si el atacante accede al SMS o email días después, el token sigue válido.
- **Historial / logs:** el token en logs del servidor o del navegador es una brecha permanente.
- **Tokens olvidados:** el usuario recuerda su contraseña y el enlace queda activo indefinidamente.
- **Ataques offline:** con tiempo ilimitado se puede atacar el hash si la BD es comprometida.

OWASP recomienda ventanas de 15 a 60 minutos. En este proyecto se implementaron 15 minutos.

1.3 Ataques en flujos de recuperación

- **Enumeración de usuarios:** Si el sistema responde diferente cuando el teléfono existe o no, un atacante puede construir listas de cuentas válidas. Contramedida: respuesta genérica idéntica en ambos casos.
- **Token predecible:** Tokens basados en `Math.random()` o timestamps pueden predecirse. Contramedida: CSPRNG (`crypto.randomBytes`).
- **Exposición de BD:** Si los tokens se guardan en texto plano, comprometer la BD permite usarlos. Contramedida: solo se almacena el hash SHA-256 del token.
- **Reutilización de token:** Un atacante que capture el token podría usarlo tras el reset legítimo. Contramedida: `flag used = TRUE` al ser consumido.
- **Race condition:** El usuario solicita reset; el atacante solicita otro; el primer token sigue activo. Contramedida: invalidar todos los tokens previos del usuario al generar uno nuevo.
- **Session fixation:** Sesiones secuestradas del usuario siguen activas aunque cambie su contraseña. Contramedida: revocar todas las sesiones activas del usuario al resetear.

1.4 Diseño accesible para recuperación de acceso

La accesibilidad en este contexto tiene dos dimensiones:

- **WCAG 2.1:** Accesibilidad técnica (WCAG 2.1): labels explícitos con <label for>, aria-describedby vinculando campos a sus mensajes de error, role="alert" y aria-live para lectores de pantalla, aria-label en botones de icono, contraste de color $\geq 4.5:1$ y errores expresados con texto + icono (no solo color).
- **UX:** Accesibilidad UX: mensajes claros y no técnicos, flujo de 3 pasos delimitados, sin revelar si el teléfono existe, confirmación explícita con redirección automática, botón "volver" en cada pantalla y toggle de visibilidad de contraseña.

2. ¿Para qué?

La recuperación de contraseña es uno de los puntos más atacados en sistemas reales, precisamente porque suele implementarse rápido y sin considerar las implicaciones de seguridad. Esta actividad tiene tres propósitos:

- **1. Proteger flujos críticos:** entender que un enlace de recuperación mal diseñado puede ser tan peligroso como no tener autenticación.
- **2. Equilibrar seguridad y UX:** un sistema ultra-seguro que el usuario no puede usar no sirve; un sistema usable pero inseguro es una brecha.
- **3. Pensar como ingeniero de seguridad:** anticipar ataques antes de que ocurran, aplicar defensa en profundidad y principio de mínima exposición.

3. ¿Qué se hizo?

Se diseñó e implementó un flujo completo de recuperación de contraseñas en el proyecto Tag Human Universal (sistema de control de acceso residencial). El trabajo se organizó en tres capas:

3.1 Base de datos — Migración SQL

Se creó la tabla `password_reset_tokens` con los siguientes campos clave:

- `token_hash` TEXT — SHA-256 del token; el texto plano NUNCA se almacena.
- `expires_at` TIMESTAMP — vigencia de 15 minutos desde la creación.
- `used` BOOLEAN — garantiza uso único del token.
- `ip_address` INET — auditoría de la IP solicitante.

Archivo: `backend/db/password_reset_migration.sql`

3.2 Backend — API de recuperación

Se implementaron tres endpoints en `passwordResetController.js` y se registraron en `authRoutes.js`:

- **POST `/api/auth/forgot-password`** Recibe el teléfono, genera el token seguro, almacena su hash y responde con un mensaje genérico (anti-enumeración). En modo desarrollo devuelve el token para pruebas.
- **GET `/api/auth/validate-reset-token?token=<hex>`** Verifica que el token exista en BD, no haya expirado y no haya sido usado. El frontend lo llama antes de mostrar el formulario.
- **POST `/api/auth/reset-password`** Valida el token, actualiza el hash de contraseña con `bcrypt` (salt 10), marca el token como usado e invalida todas las sesiones activas del usuario.

3.3 Frontend — Interfaces React

Se crearon dos nuevas pantallas y se modificaron tres archivos existentes:

- **ForgotPasswordScreen.jsx (/forgot-password)** Formulario de solicitud con validación Zod (teléfono de 10 dígitos). Muestra mensaje genérico tras enviar. En desarrollo permite copiar y usar el token directamente desde la interfaz.
- **ResetPasswordScreen.jsx (/reset-password?token=<hex>)** Valida el token automáticamente al montar. Muestra pantalla de error si expiró/inválido con opción de solicitar nuevo enlace. Formulario de nueva contraseña con confirmación, toggle de visibilidad y validación en tiempo real. Redirige al login tras éxito.
- **LoginScreen.jsx** Se agregó el enlace "¿Olvidaste tu contraseña?" con ícono KeyRound.
- **App.jsx** Se registraron las rutas /forgot-password y /reset-password.
- **api/auth.js** Se añadieron las funciones requestPasswordReset, validateResetToken y resetPassword.

3.4 Flujo completo paso a paso

- 1. El usuario hace clic en "¿Olvidaste tu contraseña?" desde el login.
- 2. Ingresa su número de teléfono (10 dígitos) y envía el formulario.
- 3. El backend genera un token de 64 chars, guarda su hash SHA-256 con expiración de 15 min e invalida tokens anteriores del mismo usuario.
- 4. Se muestra el mensaje genérico. En dev, el token aparece en pantalla con botón de copiar.
- 5. El usuario navega a /reset-password?token=<hex>.
- 6. El frontend valida el token contra el backend antes de mostrar el formulario.
- 7. El usuario ingresa y confirma su nueva contraseña.
- 8. El backend verifica el token, actualiza la contraseña con bcrypt, marca el token como usado y revoca todas las sesiones activas.
- 9. Se muestra confirmación de éxito y redirección automática al login en 3 segundos.

4. Controles de seguridad implementados

Resumen de las decisiones de seguridad y su justificación:

Control	Implementación	Protege contra
Token criptográfico	<code>crypto.randomBytes(32)</code>	Predicción / fuerza bruta
Hash SHA-256 en BD	<code>createHash("sha256")</code>	Exposición de base de datos
Expiración 15 min	<code>expires_at = now() + 15 min</code>	Tokens olvidados / capturados
Un solo uso	<code>used = TRUE</code> tras consumirse	Reutilización post-reset
Invalidación de previos	UPDATE al generar nuevo token	Race condition
Revocación de sesiones	UPDATE <code>sessions.is_active=FALSE</code>	Session hijacking
Respuesta genérica	Mismo mensaje si existe o no	Enumeración de usuarios
ARIA en formularios	<code>role</code> , <code>aria-live</code> , <code>aria-describedby</code>	Accesibilidad lector pantalla
Labels explícitos	<code><label for></code> en todos los campos	Accesibilidad táctil / teclado

5. ¿Qué se aprendió?

◆ El flujo de recuperación es un vector de ataque real

No basta con que "funcione". Un enlace de recuperación mal diseñado puede ser equivalente a no tener autenticación: si el token no expira, si se guarda en texto plano, o si el mensaje revela qué cuentas existen, el sistema es vulnerable.

◆ El texto plano del token es un secreto que no debe persistir

Solo el hash SHA-256 vive en la base de datos. El token en texto plano existe únicamente en tránsito: generado en memoria, entregado al usuario, nunca almacenado. Esta decisión de diseño protege incluso ante una brecha total de la base de datos.

◆ Defensa en profundidad

Ningún control aislado es suficiente. La combinación de token criptográfico + hash en BD + expiración + uso único + revocación de sesiones crea capas de protección que deben ser vencidas todas al mismo tiempo por el atacante.

◆ Seguridad y UX no son opuestos

Un mensaje de error genérico protege contra enumeración, pero también puede mejorar la UX al no generar ansiedad en el usuario ("¿significa que mi número no está registrado?"). El diseño accesible beneficia a todos los usuarios, no solo a quienes usan lectores de pantalla.

◆ Pensar como atacante para diseñar como ingeniero

Cada decisión de implementación surgió de la pregunta: "¿cómo lo atacaría alguien?" Ese cambio de perspectiva —de implementador a adversario— es la esencia de la ingeniería de seguridad.